PART 3

Scheduling & Real-Time

21 Cron Jobs

22 WebSockets

23 Long Polling

24 Server-Sent Events

#021

Cron Jobs

WHAT IT IS

Commands scheduled to run at fixed times.

WHY IT MATTERS

Reports, cleanups, billing runs, reminders.

EXAMPLE

```
// routes/console.php
Schedule::command('invoices:send')->dailyAt(
    '02:00')->onOneServer()->withoutOverlapping();
```

#022

WebSockets

WHAT IT IS

A persistent, two-way connection between browser and server.

WHY IT MATTERS

Instant updates (chat, live boards, tracking) with low overhead.

EXAMPLE

```
class TaskMoved implements ShouldBroadcast {
    public function __construct(public Task $task) {}
    public function broadcastOn() { return new
    PrivateChannel("board.{$this->task->board_id}");
    }
}
```

#023

Long Polling

WHAT IT IS

Client sends a request; the server holds it open until there is data or a timeout, then the client immediately asks again.

WHY IT MATTERS

Near-real-time without WebSockets; works through restrictive proxies.

EXAMPLE

```
public function poll(Request $r) {
    $until = now()->addSeconds(25);
    while (now()->lt($until)) {
        if ($msgs = Message::where('id', '>',
    $r->last_id)->get()) return $msgs;
        usleep(500_000);
    }
    return response()->noContent();
}
```

#024

Server-Sent Events

WHAT IT IS

A one-way stream from server to browser over plain HTTP (text/event-stream).

WHY IT MATTERS

Simple live feeds (progress bars, notifications, AI token streaming) with auto-reconnect.

EXAMPLE

```
return response()->stream(function () {
    while (true) {
        echo "data: " . json_encode(['progress' =>
    Cache::get('import:progress')]) . "\n\n";
        ob_flush(); flush();
        sleep(1);
    }
}, 200, ['Content-Type' => 'text/event-stream',
    'Cache-Control' => 'no-cache']);
```

Found this useful?

Save it for later.

Share it with a teammate.

Full guide with all code examples:
the Backend Engineering Handbook (GitHub).